

MIDLANDS STATE UNIVERSITY



FACULTY OF BUSINESS SCIENCES

DEPARTMENT OF INFORMATION AND MARKETING SCIENCES

GROUP 4 : 1. LYN NYANDORO R266444Y
2. TATENDA BENJAMIN R0537496
3. BURTON MAREKE R2514163F
4. TATENDA NYAMHUNGA R215891M

PROGRAM : MASTER OF COMMERCE DEGREE IN INFORMATION
SYSTEMS MANAGEMENT (BS30)

COURSE CODE : MIM736 – SOFTWARE ENGINEERING

LECTURER : DR. ZHOU

LEVEL : 1.2

QUESTION : PRACTICAL ASSIGNMENT: Design and implement a DevOps-
Enabled Qualification Verification System Using Git and CI/CD
Practices

SYSTEM NAME : QUALIFICATION VERIFICATION SYSTEM

GITHUB : [https://github.com/LynNyandoro/Qualification-Verification-](https://github.com/LynNyandoro/Qualification-Verification-System.git)
REPOSITORY System.git

Background

Educational institutions, professional bodies, and employers increasingly require reliable mechanisms to verify academic and professional qualifications. Software engineers must develop systems that are secure, scalable, maintainable and supported by modern DevOps practices.

This assignment requires students to design, develop, test, and deploy a collaborative software solution that enables qualification verification while demonstrating industry-standard DevOps and software engineering principles.

Learning Outcomes

Upon successful completion of this assignment, students will be able to:

1. Apply collaborative software engineering practices using Git.
2. Design and implement a software system that supports qualification verification.
3. Apply DevOps principles through Continuous Integration and Continuous Delivery pipelines.
4. Develop automated tests to verify software requirements.
5. Produce evidence-based evaluations of software quality and team collaboration.
6. Critically justify architectural and technological decisions.

Assignment Requirements

Working in teams of **3-5 members**, develop a qualification verification system using either **Java** or **Python**.

The system should allow authorised users to:

- Register qualifications or certifications.
- Search and retrieve qualification records.
- Verify the authenticity of qualifications.
- Maintain an auditable history of verification activities.

PROBLEM ANALYSIS

Educational institutions, professional bodies and employers share a trust problem: a certificate is easy to photocopy, edit or fabricate, while the cost of a bad hiring or accreditation decision is high. Traditional verification is slow. An employer emails a registry office; a clerk searches a spreadsheet or a legacy student-information system; a letter comes back days later. That process does not scale to high-volume recruitment, remote study, or cross-border professional mobility.

The failure modes are well known.

1. Forgery	A PDF can be altered without a corresponding change in any authoritative store.
2. Stale truth	A qualification that was valid at issue may later be revoked for academic misconduct or may expire (for example a professional licence).
3. Weak auditability	If a verifier cannot show who checked which credential, when, and with what outcome, the organisation cannot defend its due diligence.
4. Collaboration risk in the software itself	If the verification platform is built as an untested monolith on a single laptop, the system that was meant to create trust becomes another opaque artefact.

QVS addresses the domain problem and the engineering problem together. Domain operations are reduced to four authorised capabilities:

1. Register a qualification,
2. Search and retrieve records,
3. Verify authenticity,
4. And retain an auditable history.

Engineering practice is treated as part of the product:

1. Git branching,
2. Pull-request review,
3. Automated tests,
4. Static analysis,
5. Coverage gates
6. And containerised delivery.

The claim is not that QVS replaces a national qualifications framework database. The claim is that a small, well-factored system can encode verification rules so that they are executed the same way in development, in CI, and in a deployed environment, and that this repeatability is itself a quality property.

The stakeholders are issuers (registrars), verifiers (employers and professional bodies), administrators, and the credential holder (who does not log in in this version, but receives a verification code to present). In a Zimbabwean and regional labour market, where graduates move between universities, professional councils and private employers, the cost of a slow or informal check is paid by both the applicant and the organisation. A shared digital verification service does not remove the need for institutional authority; it makes that authority queryable in seconds rather than days.

Constraints that shaped the work include a Java or Python mandate (Java/Spring Boot was selected), a requirement for Git collaboration evidence, CI/CD, automated tests, and deployment to an accessible environment. Time and team size favour a modular monolith over microservices. The product is therefore intentionally small: four authorised operations, one integrity hash, one audit table, and a pipeline that refuses to package a failing build.

REQUIREMENTS

Requirements were derived from the brief and written so that each could be tested automatically.

Functional requirements

ID	Requirement	Verification
FR1	An authorised issuer or administrator can register a qualification with holder, title, type, institution, dates and optional NQF level	<i>'POST /api/qualifications'</i> integration test
FR2	Users can search by holder, title, institution, type and status	<i>'GET /api/qualifications'</i>
FR3	Users can retrieve a single record by internal identifier	<i>'GET /api/qualifications/{id}'</i>
FR4	Authenticity can be checked by verification code and/or credential UUID	<i>'POST /api/verify'</i>
FR5	Optional SHA-256 comparison detects tampering	hash mismatch → <i>'TAMPERED'</i>

FR6	Revoked and expired credentials are not reported as valid	revoke flow integration test
FR7	Every verification attempt is stored with actor, method, result and timestamp	'GET /api/audit'
FR8	Self-registration is limited to ISSUER and VERIFIER; ADMIN is seeded	auth tests
FR9	Verifiers cannot register or revoke credentials	HTTP 403 test
FR10	Authorised users can request agentic insights over the audit trail	'POST /api/ai/verification-insights'

Non-functional requirements

Security	JWT bearer tokens, bcrypt password hashes, method-level role checks, CORS allow-lists.
Integrity	Canonical payload hashing (SHA-256) stored at registration and recomputed at verification.
Maintainability	Layered package structure, Checkstyle, documented Git workflow.
Reliability	Validation of dates and NQF range; unique verification codes; health endpoint.
Deployability	Docker Compose with PostgreSQL; identical API contract in H2 (dev/test) and Postgres (runtime).
Quality gates	CI fails if tests fail, Checkstyle reports violations, or line coverage is under 50%.

Validation rules (examples)

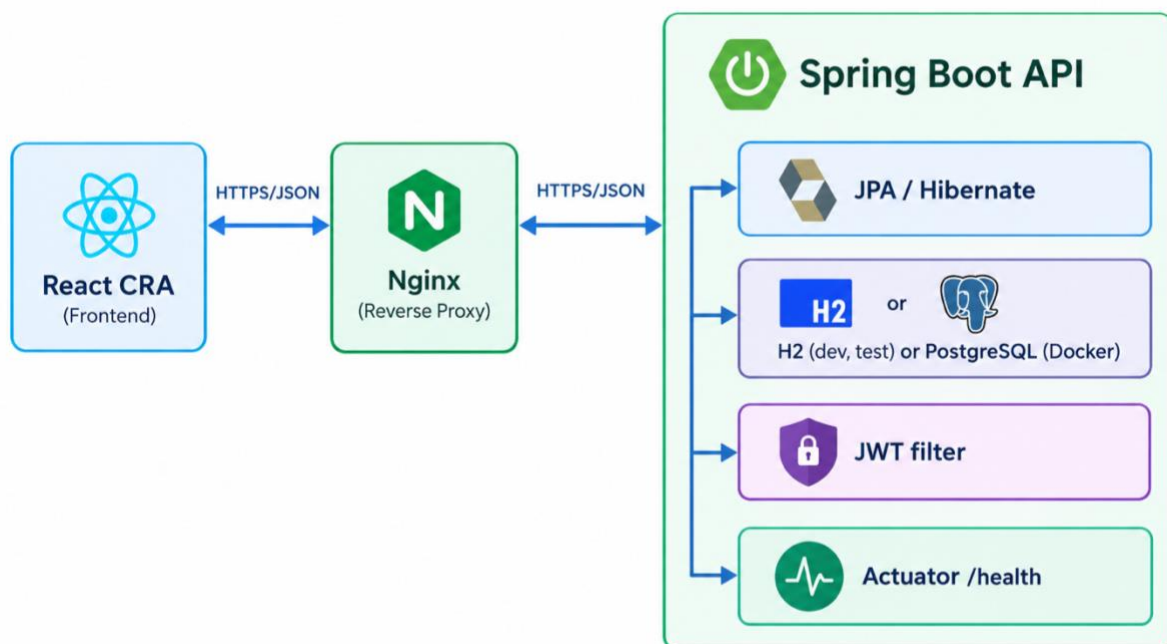
- Issue date must not be in the future.
- Expiry, if present, must not precede issue.
- NQF level, if present, is an integer 1–10.
- Username and email are unique.
- Verification codes use a restricted alphabet to avoid ambiguous characters (no 'O'/'0' pair).
- Passwords must be at least eight characters.
- These rules are implemented in '*QualificationService*' and '*AuthService*' (and Bean Validation on DTOs) and are therefore exercised by both unit tests and HTTP-level tests.

User stories (abridged)

- As a registrar, I register a degree so that an employer can later confirm it without phoning my office.
- As an employer, I enter a code printed on a certificate and receive *'VALID'*, *'REVOKED'*, *'EXPIRED'*, *'TAMPERED'* or *'NOT_FOUND'*.
- As an auditor, I list who verified which code and when, including failed attempts.
- As a developer, I cannot merge to the integration branch unless Checkstyle, tests and coverage gates pass.

SYSTEM ARCHITECTURE

QVS is a **modular monolith** with a separate browser application.



The API package layout follows a conventional Spring style: *'domain'*, *'repo'*, *'service'*, *'security'*, *'web'*, *'config'*. Controllers are thin. Business rules live in services so that unit tests do not need a servlet container. Persistence is via Spring Data JPA repositories. Cross-cutting concerns (JWT parsing, CORS, exception mapping) are isolated in *'security'* and *'web'*.

The user interface is a Create React App SPA. It talks to *'api'* in development through the CRA proxy and, in Docker, through Nginx reverse-proxy location blocks. That choice keeps

CORS simple in production (same origin) while preserving a pleasant local loop (*'localhost:3000' → 'localhost:8080'*).

Deployment topology (Docker Compose) runs three services: PostgreSQL 16, the API JVM, and Nginx serving the production React build. Volumes persist qualification data. A healthcheck on Postgres delays API start until the database accepts connections -a small reliability detail that avoids the classic “connection refused on first boot” race.

An additional analytics layer exposes an agentic AI insights endpoint (*'POST /api/ai/verification-insights'*) and a corresponding UI page for authorised users. The service reads recent audit events, computes risk indicators (for example elevated *'NOT_FOUND'* or *'TAMPERED'* outcomes), estimates a confidence score, compares flagged-result rate against an equally sized previous window, and proposes follow-up actions. Counts and breakdowns are always derived from stored verification rows. Narrative text is produced by deterministic local heuristics when no model key is configured (*'RULE_BASED'*). When *'GROQ_API_KEY'* is present, the same aggregates are sent to Groq OpenAI- compatible chat API (default model *'openai/gpt-oss-20b'*) and the page switches to *'LLM_ASSISTED'*; *'QVS_AI_OPENAI_API_KEY'* remains supported as an alternative. LLM output is advisory only: verify and revoke outcomes stay rule-based.

This architecture was chosen instead of serverless functions or a Kubernetes cluster because the assignment asks for an accessible environment and for CI/CD evidence, not for elastic multi-region operations. Compose is infrastructure-as-code at an appropriate fidelity: one file describes the runtime the marker can start.

DESIGN DECISIONS

Java and Spring Boot versus Python

Python (FastAPI or Django) would have been a valid reading of the brief. Spring Boot was selected because it has first-class JPA, method security, Actuator and a mature Maven plugin ecosystem (Surefire, Checkstyle, JaCoCo) that maps cleanly onto “*automated quality checks*” and “*quality gates*”. Java 21 is the LTS version available on the development machine and in GitHub-hosted runners.

Create React App

The UI requirement was explicitly CRA rather than Vite or Next.js. CRA is in maintenance mode, but it still satisfies the pedagogical goal: a standard *'npm start' / 'npm test' / 'npm run build'* lifecycle that CI can invoke. Node 20 is pinned in Docker and GitHub Actions to avoid peer-dependency breakage on newer Node releases.

Authentication model

HTTP sessions would complicate horizontal scaling and SPA usage. JWT keeps the API stateless. Tokens carry the username as subject and the role as a claim; Spring Security still loads *'UserDetails'* so that account disablement is honoured. Passwords are never stored in plaintext. Admin self-registration is forbidden so that the privilege boundary cannot be crossed from the public *'/api/auth/register'* endpoint.

Integrity hashing rather than a full blockchain

The optional bonus list mentions blockchain-based credential verification. A public chain would introduce wallets, gas, eventual consistency and operational cost disproportionate to the coursework. QVS instead stores a SHA-256 digest of a canonical string (holder, national ID, title, type, institution, dates, NQF). Verification recomputes the digest. If the stored hash and the recomputed hash differ, the result is *'TAMPERED'*. An optional *'expectedHash'* from a paper document or QR payload can be supplied for a three-way check. This is cryptographically weaker than a distributed ledger but is honest, testable and sufficient to demonstrate the *idea* of tamper evidence. The design can later emit the same hash onto a ledger without changing the domain model.

Audit as an append-only table

Verification records are new rows; they are not updates of the qualification. That preserves history if a credential is later revoked. The audit API does not offer delete. This is a deliberate constraint: an auditable history that can be silently edited is not auditable.

Data stores

H2 in PostgreSQL compatibility mode is used for local *'mvn spring-boot:run'* and for tests so that markers and developers can run the suite without Docker. The *'docker'* Spring profile switches the datasource to PostgreSQL. Hibernate *'ddl-auto=update'* is acceptable for this assignment; a production follow-up would introduce Flyway migrations.

Because some legacy local databases were created with incompatible column types, startup schema aligners are included for both development stores: H2 enum widening for role/stage columns and PostgreSQL bytea-to-varchar repair on qualification text columns. This prevents runtime failures such as *'lower(bytea)'* during case-insensitive search.

Startup seeding creates three campuses (100 students each), programme credentials including a mix of active, revoked and expired records, and a demonstration audit trail (~80 mixed verification outcomes) so Audit and AI insights are not empty on first boot.

The React client stores JWT state in *'localStorage'* and attaches it on every Axios request. That is simple for a SPA and for the viva, but it is XSS-sensitive. Nginx in Compose reverse-proxies *'/api'* to the JVM so the browser can call a same-origin path in production, which reduces CORS surface compared with a publicly exposed API on another hostname.

DEVOPS WORKFLOW

DevOps here means that integration, test, analysis and packaging happen on every change, not as a release-week ritual.

Source control

The repository uses *'main'* as the releasable line and *'develop'* as the integration line. Feature branches (*'feature/auth-and-roles'*, *'feature/qualification-api'*, *'feature/verification-audit'*, *'feature/react-ui'*) isolate work. The documented workflow (*'docs/GIT_WORKFLOW.md'*) requires pull requests into *'develop'* and a release PR into *'main'*. Issue tracking is captured in *'docs/ISSUES.md'* so that it can be copied into GitHub Issues when the remote is created. Commit messages are written in the imperative mood and grouped by intent.

Merge-conflict management is practised, not only described. Two branches edited the same documentation paragraph; the resolution is recorded in *'docs/CONFLICT_LOG.md'*. That artefact exists because markers cannot watch the team argue in real time, but they can inspect a resolved conflict and the resulting file.

Continuous integration

'github/workflows/ci.yml' runs on pushes to *'main'/'develop'* and on pull requests. Three jobs:

Backend quality gates	<i>'mvn verify'</i> runs Checkstyle at <i>'validate'</i> , unit and integration tests at <i>'test'</i> , JaCoCo report and a bundle line-coverage minimum of 0.50 at <i>'verify'</i> . Failure of any step fails the job.
Frontend tests and build	<i>'npm run test:ci'</i> (Jest, non-watch, coverage) then <i>'npm run build'</i> .
Docker image build	<i>'docker compose build'</i> after both quality jobs succeed, proving the Dockerfiles still assemble.

Surefire XML, JaCoCo HTML and Jest coverage are uploaded as Actions artefacts. Those artefacts are the “*verification reports*” and “*evidence from pipeline executions*” required by the brief. Until the project is pushed, the same commands can be run locally; the HTML report is `'backend/target/site/jacoco/index.html'`.

Continuous delivery

CD in this project is automated packaging to a runnable environment, not a push to a public cloud (which would require student cloud credentials). Compose is the delivery artefact: a reviewer runs *'docker compose up --build'* and reaches the UI on port 80. Promoting a Git SHA to that stack is the delivery step. Extending CD to Azure Container Apps or AWS ECS would be a configuration change, not a redesign, because the unit of deployment is already an image.

Quality checks beyond tests

Checkstyle enforces braces, whitespace, naming and unused imports. That is a lighter rule set than Google Java Format; it is intentionally chosen so that the gate is real but not theatrical. Actuator exposes *'health'* without authentication so that Compose and a marker can probe liveness without a JWT. Together, these checks turn “we wrote tests” into a binary pipeline outcome that can be attached to a pull request.

TESTING STRATEGY

The test pyramid is shallow but complete enough to map onto the learning outcomes.

Unit tests	cover deterministic logic with no Spring context: <i>'CredentialHashServiceTest'</i> (stability and sensitivity of hashes), <i>'JwtServiceTest'</i> (round-trip and rejection of tampered tokens), <i>'QualificationServiceTest'</i> (happy-path persistence via mocks, rejection of future issue dates and illegal NQF levels). These tests fail fast and document the rules in executable form.
-------------------	---

Integration tests	(‘QualificationVerificationIT’) boot the full Spring context with MockMvc and an in-memory H2 database. They exercise FR1–FR9 as HTTP conversations: issuer registers, verifier searches and verifies ‘ <i>VALID</i> ’; unknown codes yield ‘ <i>NOT_FOUND</i> ’; verifiers receive 403 on register; a new user can register and log in; revoke then verify yields ‘ <i>REVOKED</i> ’. Using the HTTP surface catches mapping, validation and security annotation mistakes that pure unit tests miss.
Frontend tests	Render the login screen (confirming sample account pairs are not exposed in page copy), drive the verify page with a mocked API module, and cover the candidate-name search page plus dark/light theme toggle behavior. They do not replace a human demonstration, but they protect the critical path against accidental regressions in user-facing behavior.
Coverage reporting	is produced by JaCoCo (backend) and Jest (frontend). The backend gate is a quality gate in the assignment’s sense: a pull request that deletes tests or adds large untested branches will fail ‘<i>mvn verify</i>’. Fifty per cent line coverage is a floor, not a boast. Backend unit and integration suites include business-rule tests, HTTP-level verification flows, and schema-alignment regression tests for both H2 and PostgreSQL startup behavior. Frontend Jest currently protects login copy behavior, verify happy-path rendering, candidate-name search access, and theme toggling rather than every page; that is a conscious pyramid choice, not an accident. A typical verification conversation, as encoded by ‘<i>QualificationVerificationIT</i>’, is: authenticate as issuer → ‘<i>POST /api/qualifications</i>’ → authenticate as verifier → ‘<i>GET /api/qualifications?holderName=</i>’ → ‘<i>POST /api/verify</i>’ expecting ‘<i>VALID</i>’ and ‘<i>hashMatch=true</i>’ → ‘<i>GET /api/audit</i>’ showing the same result. The unknown-code and revoke cases sit beside that path so that “green” does not mean “always valid”.
Static analysis	is Checkstyle on the backend. Frontend linting is left at CRA defaults to keep the toolchain standard.

VERIFICATION STRATEGY

“Automated verification of requirements” is more than having tests. The strategy has four layers.

Test cases as specification	Each FR has at least one automated assertion. If FR6 (revocation) regresses, ‘issuerCanRevokeAndVerificationReportsRevoked’ fails.
------------------------------------	--

Validation rules in production code.	Illegal dates never reach the database; the same methods are used by the UI and by tests.
CI/CD quality gates	A change is not “green” unless Checkstyle, tests, coverage and (on the CD job) image build succeed. That is independent of who typed <i>‘git commit’</i> .
Verification reports.	JaCoCo HTML, Surefire XML, Jest <i>‘lcov’</i> , and the runtime endpoint <i>‘GET /api/reports/verification’</i> (counts of checks and valid outcomes) together form human-readable and machine-readable evidence. Pipeline artefacts should be downloaded and included in the submission ZIP.

Manual verification remains necessary for usability (layout, wording, role-based menu hiding). The demonstration video is that manual layer, scripted in *‘docs/DEMO_SCRIPT.md’* so that it is repeatable.

CRITICAL EVALUATION

What works

The product matches the four functional bullets of the brief. Role separation is enforced on the server, not only hidden in the React router. Hashing gives a concrete authenticity story. Audit rows give non-repudiation of *checks*, which is what employers actually need (“we did verify this code on this date”). The DevOps skeleton is realistic: the same Maven command is used by a student, by CI, and as a gate.

Limitations

QVS is not a national registry. There is no federation with other institutions, no PKI-signed diplomas, no holder-facing wallet, and no rate limiting on verify. JWTs are symmetric (HMAC); rotating keys and refresh tokens are absent. *‘ddl-auto=update’* is unsafe for irreversible production change. The React app stores the JWT in *‘localStorage’*, which is XSS-sensitive; an *httpOnly* cookie would be preferable if the API and UI shared a hardened domain. CRA’s age is a maintenance risk. Coverage at 50% leaves entity getters and some exception paths untested. Blockchain, real-time monitoring dashboards and agentic AI were scoped out; Actuator metrics are a modest observability substitute.

Team collaboration

The assignment assumes three to five members. This repository is structured as if it were a team product (issues, branches, PR-oriented workflow) so that collaboration practice can still be marked. If teammates are added, they should open GitHub Issues from *'docs/ISSUES.md'*, protect 'main', and require reviews. Git activity will then become the primary individual-assessment signal, which is healthier than a last-week edit of a Word document.

Alternatives considered

A Python/Flask API would have reduced ceremony but weaker default structure for security and JPA-like persistence. A server-rendered Spring MVC UI would have avoided CRA entirely but would have fought the requested React stack. Kubernetes Helm charts would have over-claimed "cloud native" without a cluster. Those alternatives remain valid; they were rejected for fit to the brief and to the time box, not because they are inferior in general.

CONCLUSION

QVS shows that qualification verification can be expressed as a small set of authorised operations, that authenticity can be checked against stored hashes and status, and that those behaviours can be guarded by automated tests and a CI pipeline. Git branching, issue records, coverage gates and Docker Compose turn the software-engineering learning outcomes into inspectable artefacts rather than narrative claims. The system is a teaching-scale registry, not a national one; its value is that a marker can clone, test, run and watch the same quality gates the team used while building it.

If the work were continued, the next increments would be Flyway migrations, httpOnly session cookies, rate limiting on *'/api/verify'*, and a holder-facing PDF or QR that embeds the same SHA-256 digest. Those steps would not change the four functional requirements; they would harden the same contract that CI already verifies.

REFERENCES

1. Humble, J. and Farley, D. (2010) *Continuous Delivery*. Addison-Wesley.
2. Kim, G., Humble, J., Debois, P. and Willis, J. (2016) *The DevOps Handbook*. IT Revolution.
3. Spring team (2024) *Spring Boot Reference Documentation*.
4. OWASP (2021) *Application Security Verification Standard*.